

Final Coursework Report

Camfranglais

Lexical and Syntactic Analysis of Informal Urban Communication in Yaoundé

DETERMINISTIC / PUBLIC ACCESS / EVIDENCE-LED

A study of twelve supplied transcriptions using a custom regex lexer and a corpus-derived LL(1) parser. Includes all raw statements and tokens, grammar transformations, automata models, FIRST/FOLLOW, the predictive table, test traces and working-application screenshots.

AUTHOR GROUP

Name	Matricule
Kwete Ngouba Junior Rayan	ICTU20241377
Djemtchimo Noukui Bruno Jonatan	ICTU20241585
Amina Boubakary	ICTU20241870

Version 4.1 | 27 September 2026

Faculty of Information and Communication Technologies

Instructor: Engr. Tanwi Nkiamboh

Assignment due: 29 September 2026

Abstract and Evidence Statement

Camfranglais is a compiler-construction teaching application for examining informal expressions without replacing their wording with standardized French or English. The implementation combines a regular-expression tokenizer, priority-ordered lexical rules, explicit reference vocabulary, conservative language-transition clues and a table-driven LL(1) parser. It deliberately separates vocabulary recognition from membership in a selected category grammar.

This report evaluates twelve exact transcriptions supplied to the project and preserved in Collection. One controlled pass produces 81 tokens and 55 case-folded forms. The derived grammar has ten original nonterminals and 37 alternatives. Elimination of direct left recursion and left factoring produce twelve nonterminals, 39 alternatives and 47 populated predictive-table cells with no conflict. Ten statements parse completely. Two reject after five consumed tokens because their sixth tokens, `n'ais` and `alli`, are unresolved. Neither word is silently corrected.

The application is publicly accessible without accounts. Visitors may analyze and retain immutable test snapshots; Collection, grammar and recordings remain read-only. Browser screenshots in this report were captured from the deployed public application on 27 September 2026 without submitting additional tests. Historical aggregate statistics in those screenshots differ from this report's fixed twelve-statement evaluation.

Controlled evaluation	Measured result
Raw statements / original token occurrences	12 / 81
Known-category occurrences / unresolved occurrences	79 / 2
Emitted language-transition clues	26
Accepted / rejected category sequences	10 / 2
LL(1) conflicts / residual left-recursive nonterminals	0 / 0
Additional constructed boundary controls	7

Scope of the evidence

These are user-supplied transcriptions. The observed 83.3% grammar-match rate is a fit against the modelled grammar, not a language-accuracy score.

Contents

Abstract and Evidence Statement	2
1 Context, Problem and Objectives	4
2 Collection Method, Provenance and Ethics	5
3 Raw Corpus: Statements S01–S06	6
4 Raw Corpus: Statements S07–S12	7
5 Custom Lexical Specification	8
6 Finite-State Model of the Lexer	9
7 Lexicon, Category Policy and Notation	10
8 Token Tables: S01–S04	11
9 Token Tables: S05–S08	12
10 Token Tables: S09–S12	13
11 Frequency, Coverage and Observed Variation	14
12 Code-Mixing, Phrases and Linguistic Complexity	15
13 Constructing a Corpus-Derived Grammar	16
14 Grammar Transformations	17
15 Transformed Production Catalogue	18
16 FIRST and FOLLOW Sets	19
17 Complete LL(1) Predictive Table	20
18 Pushdown Automaton for the LL(1) Parser	21
19 Complete Accepted Trace: S02	22
20 Complete Rejected Traces: S09 and S10	23
21 Evaluation Results and Boundary Controls	24
22 Working Analyzer: Browser Evidence	25
23 Reference Views and Historical Statistics	26
24 Application Architecture and Public Deployment	27
25 Verification, Reproducibility and Limitations	28
26 Conclusion	29
References and Evidence Register	30

1 Context, Problem and Objectives

The assignment and the problem

CS4110 SET A asks a group of three to examine 10–15 real-life expressions heard in Yaoundé, preserving slang, code-mixing, incomplete speech and nonstandard forms. The required compiler work includes token categories and regular expressions, frequency and variation analysis, a grammar constructed from the expressions, grammar transformations, FIRST/FOLLOW, a parsing table and accepted/rejected test cases. The chosen parsing route here is LL(1); an LR automaton is not additionally required for that route.

The educational challenge is that source-code languages usually have an explicitly agreed syntax, whereas everyday speech is contextual and variable. A token such as `school` may occur within a French-framed expression. Different spellings may suggest the same intention without proving equivalence. An apparent mistake may be part of the original transcription.

Ebongue’s study distinguishes speaker-written Camfranglais from researcher transcriptions and discusses variation in spelling, syntax and morphosyntax [2]. Stored text and computed categories are observable, but a small textual sample cannot establish how a community speaks.

Operational objectives

1. Preserve all twelve supplied strings and label every token in source order.
2. Implement deterministic recognition rules and expose unknowns, ambiguous labels, language clues and corpus frequencies.
3. Derive reusable category productions from observed structures rather than hard-code a whitelist of twelve complete sentences.
4. Perform grammar transformations and compute every set/table cell.
5. Demonstrate predictive parsing, successful consumption and precise failure positions, including examples outside the supplied corpus.
6. Present an accessible public workspace with durable, immutable test evidence and no unnecessary account workflow.

Non-objectives

This is not a Camfranglais translator, speech recognizer, generative assistant, spelling corrector or semantic parser.

2 Collection Method, Provenance and Ethics

What was available

The twelve statements were supplied to the project on 22 September and saved without changing their wording. A read-only documentation snapshot was taken on 27 September. Every sentence in that snapshot was compared with the exact regression inputs, and the saved grammar was compared with the implemented corpus grammar.

The Collection snapshot contains thirteen entries: twelve Sentence entries and one approved Word annotation, `choa`. The latter is not counted as a thirteenth sentence and contributes no token to the twelve-statement analysis. All twelve topic categories are recorded as “Other”. Five sentence records contain location text and six contain an audio-reference field.

Evidence item	Status at the snapshot
Twelve original strings	Present and matched exactly to regression fixtures.
Supplied French meanings	Present; human-supplied.
Collection method	Not recorded in the saved project profile.
Manual-transcription confirmation	False; no new confirmation was supplied.
Original collection dates / collectors	Unconfirmed.
Speaker identities / consent	Not established.
Location / audio metadata	Some values exist; authenticity unconfirmed.

Responsible handling

Raw wording is kept separate from meanings, annotations and commentary. Suspected errors are discussed rather than silently corrected. The evidence snapshot contains only coursework-relevant text and metadata; account identifiers, credentials and server secrets are not included. The public test page warns visitors that submitted text will be visible and retained.

3 Raw Corpus: Statements S01–S06

The identifiers below are report-local labels, not private database identifiers. Case, apostrophes and wording are exactly those saved in Collection. French meanings are displayed only as *supplied glosses*.

ID	Exact raw statement	Supplied French meaning
S01	J'ai n'est pas tchop depuis bayar	Je n'ai pas mangé depuis longtemps.
S02	je suis kass je go nang	Je suis fatigué, je vais dormir.
S03	je suis back du school le sharp man	Je suis revenu de l'école ce matin.
S04	j'ai mban gars je yabat	J'ai échoué, mon gars, j'ai mal.
S05	elles yamo flop le kongossa	Elles perdent leur temps à commérer.
S06	go au piol shoua mes kako	Va à la maison laver mes habits.

Reading the evidence

S01 contains two contracted forms and the lexeme **tchop**. Its supplied meaning suggests eating, but the explicit lexicon classifies **tchop** as a noun. This disagreement is useful evidence of a coarse, context-insensitive category assignment.

S02 motivates an adjective complement followed by a juxtaposed clause and serial verbs. S03 combines a French frame with the reference words **back**, **school**, **sharp** and **man**; some categories are ambiguous. S04 contains a discourse-marker category, while S05 illustrates an adverb plus a complement whose supplied reference categories conflict. S06 is compatible with two successive verb-initial phrases.

These descriptions motivate category structures. They are not claims that one formal analysis exhausts the expressions' possible interpretations. In particular, supplied glosses may paraphrase rather than align word by word with the raw statement.

4 Raw Corpus: Statements S07–S12

These six records complete the fixed evaluation corpus. None of the later public test-history entries, dictionary rows or synthetic practice examples is added to the totals in this report.

ID	Exact raw statement	Supplied French meaning
S07	pere ca me chop je suis kass	Père, ça me touche, je suis épuisé.
S08	Ont a cote la light depuis le shap	On a coupé le courant depuis ce matin.
S09	shiba le price pere je n'ais pas flop les do	Baisse le prix, s'il te plaît, je n'ai pas beaucoup d'argent.
S10	mon combi came au school alli day a pied	Mon ami est venu à l'école, il est venu à pied.
S11	je falla le taco depuis bayar	Je cherche le taxi depuis longtemps.
S12	les talk free le francais	Ils parlent le français librement.

Interpretive themes

The wording and supplied glosses suggest fatigue or personal state (S07), electricity interruption (S08), bargaining (S09), school and walking (S10), transport (S11), and speaking French (S12). These are observations about the text, not verified locations. The corpus does not cover every topic suggested in the assignment, such as security, rain or fuel.

S08 retains `Ont a cote` exactly. The classifier gives `a` the existing English-function-word category rather than contextualizing it as a French auxiliary. S09 retains `n'ais`; S10 retains `alli`. Both are unresolved in the lexer and are the first points at which their category parses fail.

Sampling boundary

Twelve records satisfy the assignment's numerical range. This is a small development corpus, used to construct and exercise the grammar, not an independently sampled evaluation set for a language model.

5 Custom Lexical Specification

Segmentation by regular expression

The tokenizer recognizes Unicode word-like runs, internal apostrophes/hyphens, decimal numbers, selected punctuation and finally any remaining non-whitespace character. The final alternative is important: unsupported input is preserved for classification rather than silently discarded.

The following factored notation describes the segmentation pattern. **M** denotes the combining-mark ranges U+0300–036F, U+1AB0–1AFF, U+1DC0–1DFF, U+20D0–20FF and U+FE20–FE2F. Substituting those ranges and **L** into the definitions yields the pattern in the source; spaces around the displayed alternatives are notation, not characters to match.

```
L      = [^W\d_]+(?:[M]+[^W\d_]*)*
WORD   = L(?:['\u2018\u2019\u02bc-]L)*
TOKEN = WORD | \d+(?:[.,]\d+)? | [.,!?:;"()] | \S
```

Internal apostrophes therefore keep *J'ai*, *n'est* and *n'ais* as single tokens. The twelve supplied statements have no explicit punctuation tokens, but punctuation behavior is exercised by separate controls. Whitespace separates matches; the original full input, including its whitespace, is preserved independently of the token list.

Regex classification and fallback

```
NUMBER      ^\d+(?:[.,]\d+)?$
PUNCTUATION ^[.,!?:;"()]+$
ENGLISH_VERB_LIKE  suffix "ing" or "ed"
FRENCH_VERB_LIKE   suffix "er", "ir" or "re", length > 3
```

Only the first two rules are structural full matches applied before lookup. The suffix checks operate on normalized words *after* explicit lexical sources have failed. A `_LIKE` label is a morphological guess, not a confirmed part of speech. A word ending in an apparent verbal suffix can have a different function in context.

Deterministic pipeline

Tokenize the raw input; classify each token by the first applicable rule; retain each raw token and its chosen category; add phrase and language clues without altering the token stream; compute statistics; pass categories to the parser. There is no random model, translation provider or automatic spelling correction. The implementation is in `compiler/lexer/tokenizer.py` and `compiler/lexer/lexicon.py` [4].

6 Finite-State Model of the Lexer

The token pattern is regular, so it has an equivalent deterministic finite automaton (DFA). Figure 1 explains that pattern; the implementation executes Python’s `re` matcher, not a hand-written DFA class. A double circle accepts a *token form*, not vocabulary correctness or a complete grammatical statement.

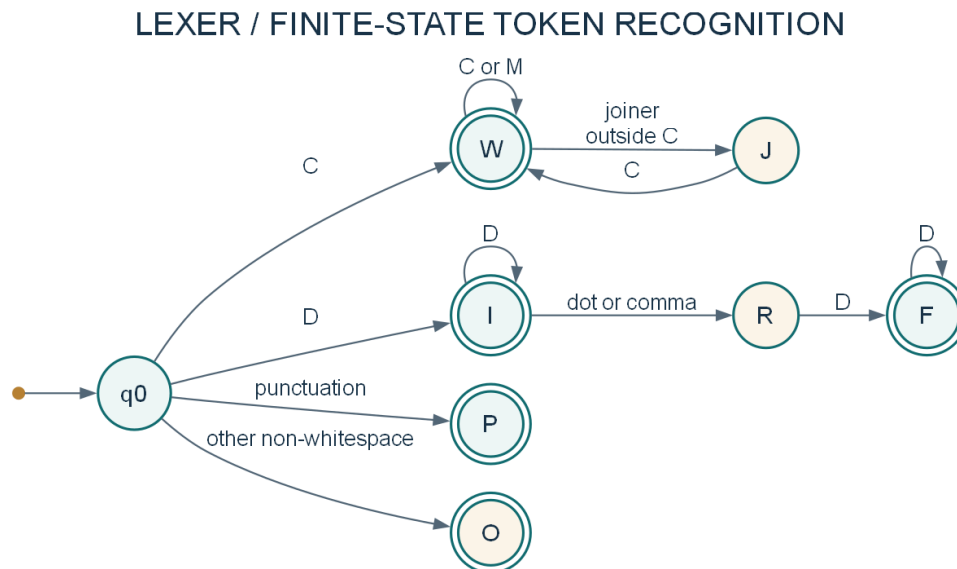


Figure 1. Equivalent token recognizer, initially in q_0 . Double-circle states accept: W word; I integer; F fraction; P punctuation; O other character. J awaits a joiner continuation and R a decimal digit; neither accepts. Unshown transitions enter an implicit dead state.

Input classes. C is $[\text{^\W\text{d}_}]$: a Unicode word character excluding decimal digits and underscore. D is $\text{\d}$; M is the combining-mark set on the preceding page. Pending joiners are ASCII/curly apostrophes or hyphen, outside C . U+02BC is already a word character and follows the C edges, even at a word boundary. Punctuation is $[.,!?:;()"]$; “other” is the remaining non-whitespace input, including a standalone underscore or hyphen.

Token boundaries. For this pattern, greedy matching retains the longest accepted prefix. A dangling joiner or decimal separator cannot finish in J or R , so the match ends at the previous accepting state and the remaining character is scanned again. Whitespace is skipped between matches, not emitted as a token. The source’s final `\S` alternative preserves other characters instead of losing them.

Raw input	Observed tokenization
j'ai	["j'ai"]
go-slow	["go-slow"]
a-	["a", "-"]
3.14	["3.14"]
3.	["3", "."]
.5	[".", "5"]
-5	["-", "5"]
x	["", "x"]

These eight constructed checks were executed through the real tokenizer. For example, `3.` is not a decimal token and `-5` is not a signed number token. A word-form path can still receive UNKNOWN during the separate priority-ordered classification stage.

7 Lexicon, Category Policy and Notation

Precedence is part of the result

After NUMBER/PUNCTUATION, a compatible approved, unambiguous Word annotation may override bundled lookup. Conflicting reviewed labels are ignored rather than arbitrarily resolved. The snapshot's approved `choa` annotation does not occur in these twelve sentences.

Explicit lookup then proceeds through SLANG, PIDGIN_MARKER, NOUN, VERB, French function words and English function words. The supplied CSV fills remaining gaps; one category is retained, multiple candidates yield AMBIGUOUS. Explicit French forms supplement gaps, then morphological guesses and finally UNKNOWN apply. Existing explicit labels take precedence over new reference labels. This explains why a dictionary origin or apparent contextual meaning does not automatically determine the emitted category.

The dictionary contains 938 display entries from 878 supplied rows plus 60 retained distinct legacy senses. Dictionary entries are source-labelled references, not collected utterances. Alias normalization can support lookup without changing displayed spelling. Multiword references annotate phrases separately; they do not assign a whole phrase's category to each component word.

Code	Lexer category	Code	Lexer category
FF	FRENCH_FUNCTION_WORD	EF	ENGLISH_FUNCTION_WORD
N	NOUN	V	VERB
ADJ	ADJECTIVE	ADV	ADVERB
AMB	AMBIGUOUS	PID	PIDGIN_MARKER
PUNC	PUNCTUATION	UNK	UNKNOWN

These compact labels are used only to keep printed tables readable. The JSON and application retain full category names. The lexer supports twenty categories overall; only nine occur in this corpus. The selected grammar uses nine terminals as well, but these are not the same nine: it includes PUNC and excludes UNK. Other supported lexer categories may legitimately fall outside this particular grammar.

Colloquial words need not receive the literal SLANG label. For example, `kass` is ADJ and `nang` is V under the current lookup. The category system is a software convention, not an exhaustive sociolinguistic taxonomy.

8 Token Tables: S01–S04

Every token below is emitted by the running lexer, in original order. Labels use the legend on the previous page; neither capitalization nor apostrophes have been normalized in the raw-token column.

S01 6 tokens

J'ai n'est pas tchop depuis bayar

No.	Raw token	Label
1	J'ai	V
2	n'est	V
3	pas	FF
4	tchop	N
5	depuis	FF
6	bayar	ADV

S02

6 tokens

je suis kass je go nang

No.	Raw token	Label
1	je	FF
2	suis	V
3	kass	ADJ
4	je	FF
5	go	V
6	nang	V

S03 8 tokens

je suis back du school le sharp man

No.	Raw token	Label
1	je	FF
2	suis	V
3	back	AMB
4	du	FF
5	school	N
6	le	FF
7	sharp	ADV
8	man	AMB

S04

5 tokens

j'ai mban gars je yabat

No.	Raw token	Label
1	j'ai	V
2	mban	V
3	gars	PID
4	je	FF
5	yabat	V

What these classifications show

S01's J'ai and n'est are whole VERB tokens, not separately parsed pronoun/auxiliary contractions. The grammar therefore operates at a different granularity from a detailed French morphosyntactic analysis.

S02's go nang yields two verbs, enabling a serial-verb tail. S03 contains two ambiguous tokens without an UNKNOWN, a direct example of recognized but unresolved reference classification. S04 labels gars as a Pidgin marker because that explicit list has priority.

The lengths are six, six, eight and five tokens respectively. These 25 occurrences contribute to the corpus total once each.

9 Token Tables: S05–S08

The same deterministic precedence is applied across statements.

S05 5 tokens
elles yamo flop le kongossa

No.	Raw token	Label
1	elles	FF
2	yamo	V
3	flop	ADV
4	le	FF
5	kongossa	AMB

S06 6 tokens
go au piol shoua mes kako

No.	Raw token	Label
1	go	V
2	au	FF
3	piol	N
4	shoua	V
5	mes	FF
6	kako	N

S07 7 tokens
pere ca me chop je suis kass

No.	Raw token	Label
1	pere	N
2	ca	FF
3	me	AMB
4	chop	PID
5	je	FF
6	suis	V
7	kass	ADJ

S08 8 tokens
Ont a cote la light depuis le shap

No.	Raw token	Label
1	Ont	V
2	a	EF
3	cote	V
4	la	FF
5	light	N
6	depuis	FF
7	le	FF
8	shap	ADV

Interpretation

S05's **flop** is ADV and **kongossa** is AMB. S06 shows that two verb-initial phrases can be adjacent without punctuation. In S07, **pere** is N, **ca** is FF and **chop** is PID. The coarse PID category can cover items with different discourse or verbal roles.

S08 illustrates a limitation of fixed lookup: **a** is EF because the explicit English-function-word set wins before contextual interpretation. The original **Ont** is V. Their resulting V–EF–V pattern is modeled as an observed category link.

These four statements contribute 26 occurrences. Along with the previous 25, they account for 51 of the 81 tokens.

10 Token Tables: S09–S12

Both rejected originals are included in full. The parser stops at the first unsupported lookahead, but the lexer still classifies every token.

S09 10 tokens

shiba le price pere je n'ais pas flop les do

No.	Raw token	Label
1	shiba	V
2	le	FF
3	price	N
4	pere	N
5	je	FF
6	n'ais	UNK
7	pas	FF
8	flop	ADV
9	les	FF
10	do	EF

S10 9 tokens

mon combi came au school alli day a pied

No.	Raw token	Label
1	mon	FF
2	combi	N
3	came	V
4	au	FF
5	school	N
6	alli	UNK
7	day	N
8	a	EF
9	pied	N

S11 6 tokens

je falla le taco depuis bayar

No.	Raw token	Label
1	je	FF
2	falla	V
3	le	FF
4	taco	N
5	depuis	FF
6	bayar	ADV

S12 5 tokens

les talk free le francais

No.	Raw token	Label
1	les	FF
2	talk	V
3	free	AMB
4	le	FF
5	francais	N

The two unresolved forms

n'ais and alli each occur once and are the sixth token in their respective statement. They match no explicit reference, approved annotation or permitted morphological fallback. Each is therefore represented as UNK.

S09's later do and S10's later a retain EF, even though parsing has already failed before them. The complete lexical analysis and the partial parser consumption are different outputs: “consumed five” does not mean “only five tokens were extracted”.

The lengths here are ten, nine, six and five, contributing the final 30 occurrences.

11 Frequency, Coverage and Observed Variation

Statistics below use each of the twelve statements exactly once. Frequencies case-fold spelling; original forms remain available in the token tables. The sample contains 81 occurrences, 55 case-folded forms and a mean of 6.75 tokens per statement. Lengths range from five to ten.

Category	Code	Count	Share	Case-folded form	Occurrences
FRENCH_FUNCTION_WORD	FF	28	34.6%	je	7
VERB	V	19	23.5%	le	6
NOUN	N	14	17.3%	depuis	3
ADVERB	ADV	6	7.4%	suis	3
AMBIGUOUS	AMB	5	6.2%	j'ai	2
ENGLISH_FUNCTION_WORD	EF	3	3.7%	pas	2
ADJECTIVE	ADJ	2	2.5%	bayar	2
PIDGIN_MARKER	PID	2	2.5%	kass	2
UNKNOWN	UNK	2	2.5%	go	2
				school	2
				flop	2
				au	2
				pere	2
				a	2
				les	2

FF and V account for 47/81 occurrences. This reflects the selected sample and the broad category definitions; it is not an estimate of category prevalence in Yaoundé. Percentages are independently rounded.

Forty forms occur once

n'est, tchop, nang, back, du, sharp, man, mban, gars, yabat, elles, yamo, kongossa, piol, shoua, mes, kako, ca, me, chop, ont, cote, la, light, shap, shiba, price, n'ais, do, mon, combi, came, alli, day, pied, falla, taco, talk, free, francais.

Coverage is not accuracy

Seventy-nine of 81 occurrences receive a non-UNKNOWN category (97.5%). That measures lookup coverage, not correct linguistic tagging: recognized AMBIGUOUS and context-insensitive labels are included. No independently annotated gold standard was supplied, so precision and recall are not computed.

The orthographic variation function uses case-folding, Unicode decomposition, accent removal and curly-apostrophe normalization to propose groups. In this sample it finds only J'ai and j'ai, one occurrence each. The pair **sharp/shap** is *not* grouped by that function because their normalized strings differ.

12 Code-Mixing, Phrases and Linguistic Complexity

Language clues are separate from part of speech. Word lists and explicit reference origins provide candidate sets for FR, EN and PID. Shared forms can have several candidates. The algorithm narrows compatible runs and emits the last anchor and new token when evidence becomes incompatible. UNKNOWN carries no language evidence; sentence-ending punctuation resets the run.

ID	Exact emitted transition endpoints
S01	pas ... tchop; tchop ... depuis
S02	je ... go
S03	suis ... back; back ... du; du ... school; school ... le; le ... sharp; sharp ... man
S04	None emitted
S05	le ... kongossa
S06	go ... au
S07	ca ... chop; chop ... je
S08	la ... light; light ... depuis; le ... shap
S09	le ... price; price ... pere; les ... do
S10	mon ... combi; combi ... came; came ... au; au ... school; day ... pied
S11	None emitted
S12	les ... talk; free ... le

There are 26 emitted clues, not 26 proven language switches. An ellipsis denotes endpoints and can span intervening tokens. No verb-phrase or slang-phrase regex matches these twelve inputs, although the raw expressions still contain colloquial words and constructions.

French framing around words such as **school**, **light** and **price** illustrates the interaction between grammar, borrowing and code-mixing. A fixed word origin cannot determine whether a speaker perceives a form as a switch or an established borrowing. Ellipsis, juxtaposition, local idioms and contextual meanings also exceed the information represented by a category sequence.

Written variation must be distinguished from pronunciation and from a transcriber's choices [2]. The available material cannot support claims about speakers' ages, education, neighborhoods, accent or intentions. S04 and S11 emit no transition clue; that absence is limited evidence, not a certificate that the originals are monolingual.

13 Constructing a Corpus-Derived Grammar

The chosen CFG models recurring lexer-category structures instead of listing twelve raw sentences. The original has ten nonterminals and 37 alternatives. For compact printing, Ut, Cl, St, Bt, Pt, Lp, Mn, At, Ct and Nm abbreviate Utterance, Clause, SubjectTail, BareSubjectTail, PredicateTail, LinkedPhrase, ModifiedNominal, AdverbTail, ComplementTail and Nominal respectively.

LHS	Original alternatives
Ut	Cl Cl PUNC
Cl	FF St Nm Bt V Pt
St	V Pt Nm V Pt AMB PID Ct
Bt	V Pt FF St
Pt	V Pt EF V Pt ADJ Ct AMB Ct ADV At Nm Ct FF Lp PID Ct ϵ
Lp	V Pt Nm Ct AMB Ct ADV At FF Mn
Mn	Nm Ct AMB Ct ADV At
At	Nm Ct AMB Ct FF Lp V Pt ϵ
Ct	FF Lp V Pt ϵ
Nm	Nm N N

Evidence-to-rule rationale

Cl permits a function-word subject, a nominal opening or a verb-initial clause. St covers *je suis*, *mon combi came* and the observed AMB–PID pattern in *ca me chop*. Pt permits auxiliary/serial verbs (*j'ai mban*, *go nang*), observed V–EF–V links, and nominal, adjectival, adverbial or ambiguous complements.

Lp and Mn represent introduced groups such as *au piol*, *depuis bayar* and *depuis le shap*. Ct allows juxtaposition, as in *je suis kass je go nang* and *go au piol shoua mes kako*. Nm represents noun clusters; its natural left recursion is intentionally processed by the real transformation routine.

Ut permits one optional punctuation token, although the twelve inputs lack one. This narrowly stated extension is tested separately. UNKNOWN is excluded, and AMB is allowed only in explicit positions. A new empty installation may use an illustrative starter grammar; that starter is not the saved grammar evaluated in this report.

14 Grammar Transformations

Step 1: eliminate direct left recursion

The implementation detects the original left-recursive nonterminal N_m :

$$N_m \rightarrow N_m N \mid N.$$

It introduces `Nominal_LR1`, abbreviated N_r , and rewrites:

$$N_m \rightarrow N N_r, \quad N_r \rightarrow N N_r \mid \epsilon.$$

Both forms generate one or more noun-category tokens. The original extends a noun chain by left recursion; the transformed form consumes the first noun and extends it through a right-recursive suffix. No raw noun is deleted and the language of this subgrammar is unchanged.

Step 2: left-factor the start rule

The original start rule has a shared prefix:

$$U_t \rightarrow C_l \mid C_l \text{ PUNC}.$$

The routine introduces `Utterance_LF1`, abbreviated U_f :

$$U_t \rightarrow C_l U_f, \quad U_f \rightarrow \epsilon \mid \text{PUNC}.$$

The alternative choice is postponed until the clause has been processed. End-of-input selects epsilon; PUNC selects the optional punctuation. The empty alternative is a suffix, not permission for an empty whole utterance.

What the implementation logged

The two operation identifiers are `eliminate_direct_left_recursion` and `left_factor`. Each record contains the complete grammar before and after the operation. No ordered indirect-substitution step was needed for this particular rule ordering. The generic implementation supports bounded ordered substitution and separately checks for residual nullable-prefix recursion.

The final result has twelve nonterminals, 39 alternatives, no residual left-recursive nonterminal and no LL(1) conflict. Transformation limits reject degenerate cycles or unsafe expansion instead of claiming success. The regression suite also compares original and transformed languages with an independent bounded derivation oracle up to four terminals; that is useful finite evidence, not a proof for every arbitrary CFG.

15 Transformed Production Catalogue

The identifiers P01–P39 are report references assigned in the computed grammar’s order. They are used consistently in the predictive table and traces. Nr is `Nominal_LR1`; Uf is `Utterance_LF1`.

ID	Transformed production	ID	Transformed production
P01	Ut → Cl Uf	P21	Lp → AMB Ct
P02	Cl → FF St	P22	Lp → ADV At
P03	Cl → Nm Bt	P23	Lp → FF Mn
P04	Cl → V Pt	P24	Mn → Nm Ct
P05	St → V Pt	P25	Mn → AMB Ct
P06	St → Nm V Pt	P26	Mn → ADV At
P07	St → AMB PID Ct	P27	At → Nm Ct
P08	Bt → V Pt	P28	At → AMB Ct
P09	Bt → FF St	P29	At → FF Lp
P10	Pt → V Pt	P30	At → V Pt
P11	Pt → EF V Pt	P31	At → ε
P12	Pt → ADJ Ct	P32	Ct → FF Lp
P13	Pt → AMB Ct	P33	Ct → V Pt
P14	Pt → ADV At	P34	Ct → ε
P15	Pt → Nm Ct	P35	Nm → N Nr
P16	Pt → FF Lp	P36	Nr → N Nr
P17	Pt → PID Ct	P37	Nr → ε
P18	Pt → ε	P38	Uf → ε
P19	Lp → V Pt	P39	Uf → PUNC
P20	Lp → Nm Ct		

How to use the catalogue

P01 starts every parse by expanding Ut into Cl and Uf. P02–P04 select a clause opening from FF, N or V. S02 then selects P05 and P12 for its verb and adjective complement. Later P32 and P19 admit the juxtaposed phrase, and P10 admits its next verb. The empty suffixes P18 and P38 complete the parse.

P35–P37 describe noun clusters. P37 can be chosen only when the next token belongs to FOLLOW(Nr), not for every unexpected token. P38 and P39 preserve the original optional single-punctuation choice. The table on the next two pages makes each choice explicit rather than relying on procedural “try alternatives until one works” logic.

The aliases and production numbers affect only presentation. Full original names, alternatives, generated helpers and transformation records are included in the companion computed JSON, so the calculations can be reproduced without reverse-engineering the abbreviations.

16 FIRST and FOLLOW Sets

FIRST contains possible initial terminals of a derivation and epsilon when the nonterminal is nullable. FOLLOW contains terminals, or the end marker \$, that can immediately follow a non-terminal in a derivation from the start symbol. Epsilon does not belong in FOLLOW.

NT	FIRST	FOLLOW
Ut	FF, N, V	\$
Cl	FF, N, V	\$, PUNC
St	AMB, N, V	\$, PUNC
Bt	FF, V	\$, PUNC
Pt	ADJ, ADV, AMB, EF, FF, N, PID, V, ϵ	\$, PUNC
Lp	ADV, AMB, FF, N, V	\$, PUNC
Mn	ADV, AMB, N	\$, PUNC
At	AMB, FF, N, V, ϵ	\$, PUNC
Ct	FF, V, ϵ	\$, PUNC
Nm	N	\$, FF, PUNC, V
Nr	N, ϵ	\$, FF, PUNC, V
Uf	PUNC, ϵ	\$

Fixed-point computation

For each production, FIRST accumulates the first symbol's terminal evidence, continuing through nullable prefixes and adding epsilon only when the entire right-hand side is nullable. The implementation repeats these updates until no set grows. FOLLOW starts by adding \$ to Ut. For each nonterminal within a right-hand side, FIRST of the suffix contributes non-epsilon terminals; a nullable or absent suffix also contributes FOLLOW of the left side. Those updates likewise continue to a fixed point.

Sanity checks on this grammar

FIRST(Ut) is exactly {FF, N, V}; hence empty input cannot begin a clause. FOLLOW(Ut) contains only end-of-input. Uf is nullable and permits punctuation, which explains PUNC in FOLLOW(Cl). Nr can begin another N or become empty, but its FOLLOW set is {\$, FF, PUNC, V}, not UNKNOWN.

These sets are computed from the transformed productions, not handwritten independently for the report. Reporting the full sets allows a reader to verify the table's epsilon entries and to explain why both unresolved sixth tokens lead to rejection.

17 Complete LL(1) Predictive Table

For $A \rightarrow \alpha$, each terminal in $\text{FIRST}(\alpha) \setminus \{\epsilon\}$ receives that production. If α is nullable, its production is also placed under every terminal in $\text{FOLLOW}(A)$. Multiple distinct productions in one cell are conflicts; the implementation retains the conflict and does not choose an arbitrary winner.

The complete table is printed below. Columns are the nine grammar terminals plus end-of-input. The 47 populated cells refer to P01–P39 in the catalogue. “–” means no production and causes rejection for that nonterminal/lookahead.

NT	ADJ	ADV	AMB	EF	FF	N	PID	PUNC	V	\$
Ut	–	–	–	–	P01	P01	–	–	P01	–
Cl	–	–	–	–	P02	P03	–	–	P04	–
St	–	–	P07	–	–	P06	–	–	P05	–
Bt	–	–	–	–	P09	–	–	–	P08	–
Pt	P12	P14	P13	P11	P16	P15	P17	P18	P10	P18
Lp	–	P22	P21	–	P23	P20	–	–	P19	–
Mn	–	P26	P25	–	–	P24	–	–	–	–
At	–	–	P28	–	P29	P27	–	P31	P30	P31
Ct	–	–	–	–	P32	–	–	P34	P33	P34
Nm	–	–	–	–	–	P35	–	–	–	–
Nr	–	–	–	–	P37	P36	–	P37	P37	P37
Uf	–	–	–	–	–	–	–	P39	–	P38

Worked entries

- $M[\text{Ut}, \text{FF}] = \text{P01}$, since Cl can begin with FF and Uf follows it. The same production is selected under N and V.
- $M[\text{Pt}, \text{ADJ}] = \text{P12}$, selecting $\text{Pt} \rightarrow \text{ADJ Ct}$.
- $M[\text{Pt}, \$] = \text{P18}$, since the Pt alternative is empty and end-of-input is in $\text{FOLLOW}(\text{Pt})$.
- $M[\text{Nr}, \text{N}] = \text{P36}$, whereas FF, V, PUNC and \$ select its empty alternative P37.
- Uf selects P38 at \$ and P39 at PUNC. No row has a defined UNKNOWN entry.

LL(1) conclusion

No cell has competing productions, and the transformed grammar has no residual left recursion. Predictive parsing is therefore enabled for this grammar. A valid everyday statement can still have a category sequence outside the modeled subset.

18 Pushdown Automaton for the LL(1) Parser

The parser is a *bounded implementation of a deterministic pushdown recognizer*, not another token DFA. Its extra memory is the symbol stack. For the saved grammar, the stack alphabet contains twelve nonterminals, nine grammar terminals and $\$$. Input consists of lexer categories, not raw words; a virtual $\$$ marks the end.

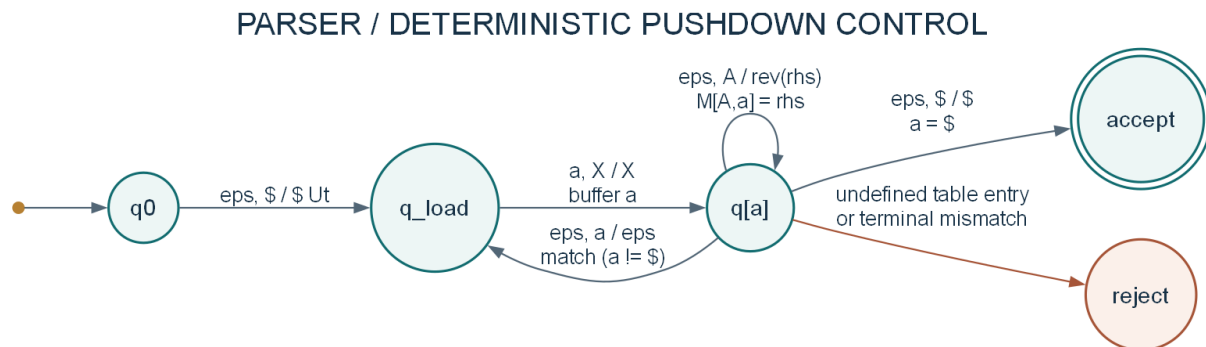


Figure 2. Finite control of the predictive pushdown automaton. An edge label is “input, stack top / replacement”; **eps** consumes no input. The stack top is at the right. $q[a]$ abbreviates one control state per possible buffered lookahead, not an unbounded variable.

Expand, match, accept. The initial stack becomes $[\$, Ut]$. With a nonterminal A on top, the unique table entry $M[A, a]$ replaces it by the reversed right-hand side without consuming the buffered category. An epsilon production only pops A . If a terminal matches a , it is popped and the next lookahead is loaded. Acceptance requires $\$$ on top and at input end; the implementation’s final trace still shows the bottom marker rather than an empty stack.

Why this is deterministic. Loading reads one category into finite control; expansion and matching then use epsilon transitions. The table has no conflicting entry, so each valid configuration selects at most one action. There are twenty supported lexer categories plus the end marker. The diagram uses a prefetched lookahead; the actual code instead reads `categories[position]` and advances `position` only on a match. This equivalent presentation does not change the accepted category sequences.

Concrete execution. In S02, the first expansion changes $[\$, Ut]$ to $[\$, Uf, C1]$. Later matching FF consumes the first token. S09/S10 instead reach an undefined table entry with UNK after five matches, so neither the unknown nor the following tokens is skipped. The complete traces on the next pages show the recorded configurations.

Implemented guard	Bound
Input tokens	256
Characters in an individual token	1,000
Recorded parse steps	2,000
Stack symbols	1,024

Malformed tokens and non-LL(1) grammars are rejected before the stack loop. The implementation also checks repeated configurations and the bounds above. Trace snapshots precede each action.

19 Complete Accepted Trace: S02

Input: je suis kass je go nang. Categories: FF, V, ADJ, FF, V, V. The table below contains every recorded step; production identifiers refer to the full catalogue. Each stack/remaining pair precedes its action.

Step	Stack (top at right)	Remaining input	Action
1	\$ Ut	FF V ADJ FF V V \$	P01
2	\$ Uf Cl	FF V ADJ FF V V \$	P02
3	\$ Uf St FF	FF V ADJ FF V V \$	match FF
4	\$ Uf St	V ADJ FF V V \$	P05
5	\$ Uf Pt V	V ADJ FF V V \$	match V
6	\$ Uf Pt	ADJ FF V V \$	P12
7	\$ Uf Ct ADJ	ADJ FF V V \$	match ADJ
8	\$ Uf Ct	FF V V \$	P32
9	\$ Uf Lp FF	FF V V \$	match FF
10	\$ Uf Lp	V V \$	P19
11	\$ Uf Pt V	V V \$	match V
12	\$ Uf Pt	V \$	P10
13	\$ Uf Pt V	V \$	match V
14	\$ Uf Pt	\$	P18
15	\$ Uf	\$	P38
16	\$	\$	accept

Explanation of the derivation

P02 chooses the FF-led clause. After matching **je**, P05 provides a verb and predicate tail. P12 selects an adjective complement for **kass**. P32 consumes the next FF and P19 allows a following verb, modeling the juxtaposition rather than insisting on punctuation or a conjunction.

P10 permits the second verb in **go nang**. Once all six input categories have been consumed, P18 removes the empty predicate tail and P38 removes the optional punctuation suffix. The final row sees only the two end markers and accepts. The consumed count is six and the error field is null.

What acceptance means

This is evidence that the observed category pattern belongs to the transformed grammar. Vocabulary approval also passes because no UNKNOWN occurs, but that is a separate calculation. The contrast becomes visible in the constructed known-word controls, where vocabulary passes and CFG parsing rejects.

20 Complete Rejected Traces: S09 and S10

The lexer completes both inputs. The parser then rejects at the first unavailable table entry; no recovery production skips the unknown.

S09: n'ais at position 6.

Step	Stack (top at right)	Remaining input	Action
1	\$ Ut	V FF N N FF UNK FF ADV FF EF \$	P01
2	\$ Uf Cl	V FF N N FF UNK FF ADV FF EF \$	P04
3	\$ Uf Pt V	V FF N N FF UNK FF ADV FF EF \$	match V
4	\$ Uf Pt	FF N N FF UNK FF ADV FF EF \$	P16
5	\$ Uf Lp FF	FF N N FF UNK FF ADV FF EF \$	match FF
6	\$ Uf Lp	N N FF UNK FF ADV FF EF \$	P20
7	\$ Uf Ct Nm	N N FF UNK FF ADV FF EF \$	P35
8	\$ Uf Ct Nr N	N N FF UNK FF ADV FF EF \$	match N
9	\$ Uf Ct Nr	N FF UNK FF ADV FF EF \$	P36
10	\$ Uf Ct Nr N	N FF UNK FF ADV FF EF \$	match N
11	\$ Uf Ct Nr	FF UNK FF ADV FF EF \$	P37
12	\$ Uf Ct	FF UNK FF ADV FF EF \$	P32
13	\$ Uf Lp FF	FF UNK FF ADV FF EF \$	match FF
14	\$ Uf Lp	UNK FF ADV FF EF \$	reject

S10: alli at position 6.

Step	Stack (top at right)	Remaining input	Action
1	\$ Ut	FF N V FF N UNK N EF N \$	P01
2	\$ Uf Cl	FF N V FF N UNK N EF N \$	P02
3	\$ Uf St FF	FF N V FF N UNK N EF N \$	match FF
4	\$ Uf St	N V FF N UNK N EF N \$	P06
5	\$ Uf Pt V Nm	N V FF N UNK N EF N \$	P35
6	\$ Uf Pt V Nr N	N V FF N UNK N EF N \$	match N
7	\$ Uf Pt V Nr	V FF N UNK N EF N \$	P37
8	\$ Uf Pt V	V FF N UNK N EF N \$	match V
9	\$ Uf Pt	FF N UNK N EF N \$	P16
10	\$ Uf Lp FF	FF N UNK N EF N \$	match FF
11	\$ Uf Lp	N UNK N EF N \$	P20
12	\$ Uf Ct Nm	N UNK N EF N \$	P35
13	\$ Uf Ct Nr N	N UNK N EF N \$	match N
14	\$ Uf Ct Nr	UNK N EF N \$	reject

S09 ends at Lp (LinkedPhrase) with lookahead UNK; its legal lookaheads are ADV, AMB, FF, N and V. S10 ends at Nr (Nominal_LR1) with UNK; legal lookaheads are \$, FF, N, PUNC and V. Both have consumed five tokens. The different stack tops explain the distinct error messages even though the failure positions match.

21 Evaluation Results and Boundary Controls

All twelve exact originals were evaluated using the same saved grammar and reviewed lexicon snapshot. Each lexical token, parser result, consumed count and trace length below is computed.

ID	Tokens	Vocabulary	CFG	Consumed	Trace rows
S01	6	Pass	Accept	6	18
S02	6	Pass	Accept	6	16
S03	8	Pass	Accept	8	22
S04	5	Pass	Accept	5	14
S05	5	Pass	Accept	5	14
S06	6	Pass	Accept	6	20
S07	7	Pass	Accept	7	19
S08	8	Pass	Accept	8	21
S09	10	Fail	Reject	5	14
S10	9	Fail	Reject	5	14
S11	6	Pass	Accept	6	18
S12	5	Pass	Accept	5	16

Ten statements fit the grammar (83.3%); two reject (16.7%). The following constructed controls demonstrate why the verdicts must nevertheless remain separate. These are software tests, not additional collected statements.

ID	Constructed control	Vocabulary	CFG
C01	(empty)	Pass	Reject
C02	school	Pass	Reject
C03	je	Pass	Reject
C04	je suis kass.	Pass	Accept
C05	je suis kass?!	Pass	Reject
C06	je je suis	Pass	Reject
C07	je suis alli	Fail	Reject

Empty input has no UNKNOWN but cannot derive a Clause. A noun-only fragment and a dangling FF-led clause also reject. One punctuation token is supported; two are not. Known categories in an unsupported order can reject without any unknown word.

The grammar-development corpus is also the principal demonstrated corpus. Regression tests add unseen combinations, negative category orders and bounded language-equivalence checks, but a larger consented corpus with independent annotation would be necessary for stronger linguistic evaluation.

22 Working Analyzer: Browser Evidence

The captures were taken from <https://camfranglais.duckdns.org> on 27 September 2026. They show the implemented public interface. The documentation session opened an existing saved test rather than submitting another test.

SELECTED SAVED TEST

Analyzed sentence or word

Recorded 9/25/2026, 7:49:11 AM. These are the original results, not a new analysis using today's settings.

Publicly retained original text and results. Saved tests are immutable.

je suis kass je go nang

ACCEPT

No UNKNOWN tokens. Recognized slang and all other known lexer categories are accepted.

Vocabulary approval is not grammatical correctness. The CFG grammar check is shown separately in Analysis.

Use as analyzer input →

▼ Token details for this test

#	Observed text	Lexer category
1	je	FRENCH_FUNCTION_WORD
2	suis	VERB
3	kass	ADJECTIVE
4	je	FRENCH_FUNCTION_WORD
5	go	VERB
6	nang	VERB

Screenshot. Existing S02 selected through Analysis, with its original input, vocabulary verdict and token details.

Franc Analyzer

Split into tokens, classify using the shared vocabulary, then test the saved grammar.

Statement to analyze

Type or paste a sentence or word. Leave blank to test empty input (epsilon).

0 / 4,000 characters · not saved to Collection

<> Analyze

Using saved grammar

Analyze publicly retains this test's original text, saved grammar and results, even when rejected. Do not submit personal or confidential content. Repeated completed tests count separately. Nothing is added to Collection.

Camfranglais · Compiler Workspace
Data & privacy →

Screenshot. The current analyzer opens directly without login. Submission explicitly warns that tests are public and retained. This crop shows the input panel.

Inspecting S02 retrieves its original token table and trace, not a new analysis. The uncropped captures and metadata are supplied for verification.

23 Reference Views and Historical Statistics

Workspace / Collection

Camfranglais

Compiler workspace

WORKSPACE

Franc Analyzer

Analysis

Collection

Dictionary

Synthetic examples

Public workspace

Analysis & save tests - Reference data read-only

Privacy information

French - English - Francanglais - Pidgin

Workspace / Collection

Compiler connected

READ-ONLY REFERENCE COLLECTION

Collection

Browse saved statements, annotations and provenance, and listen to existing recordings. Entries cannot be added, edited or removed here.

13 Total entries

7 Approved

6 Awaiting review

Counts cover the whole collection, not just the search results.

Use a record as analyzer input without changing Collection. Dictionary and synthetic references are not fieldwork evidence.

Search statements, words, meanings or context

All languages

All review statuses

All categories

All types

Collected records 13 entries

Export results (JSON)

Exports include the displayed entries and their context/contributor details. Share only with authorized recipients.

Word Other

Camfranglais Approved

choa

Public reference record - Read-only

prendre quelque chose par la force

taking something using power

Sentence Other

Camfranglais Approved

les talk free le francais

Public reference record - Read-only

ils parlent le français librement.

i speak well in french

Screenshot. The Collection view is read-only and distinct from saved test history. Raw statements and supplied metadata remain viewable.

Workspace / Analysis

Camfranglais

Compiler workspace

WORKSPACE

Franc Analyzer

Analysis

Collection

Dictionary

Synthetic examples

Public workspace

Analysis & save tests - Reference data read-only

Privacy information

French - English - Francanglais - Pidgin

Workspace / Analysis

Compiler connected

Analysis

Statistics and original results from all publicly saved tests.

Back to Franc Analyzer

Grammar settings

PUBLIC WORKSPACE / RETAINED TESTS

Test statistics

All completed public tests, including repeated inputs. Original text and results remain after refresh. Untested Collection entries are not included.

Tests recorded: 34

Accepted: 24

Rejected: 10

Acceptance rate: 70.6%

These are vocabulary results: a test is rejected only when its saved categories include UNKNOWN. Recognized slang, ambiguous known words and other recognized categories pass. Empty input also has no UNKNOWN tokens. Original tokens and parser results are never rewritten.

Separate CFG grammar checks

Matched grammar: 15

Did not match grammar: 19

Grammar match rate: 44.1%

Total tokens: 179

Raw spellings: 72

Normalized forms: 70

Unknown occurrences: 14

Grammar checks measure each test against the rules used at that time, not the correctness of a speaker's language. They do not decide vocabulary approval. Changing grammar settings does not rewrite past results.

Screenshot. The deployed Analysis view reports all 34 saved tests observed at capture: 24 vocabulary passes and ten failures; separately, 15 grammar matches and 19 mismatches.

The history screenshot totals 179 tokens and includes repeated inputs, additional public tests and older lexical/grammar snapshots. Its vocabulary rate (70.6%) and grammar-match rate (44.1%) are therefore not the fixed twelve-statement result of this report. Older snapshots can retain a former UNKNOWN classification even after the current lexer recognizes that word. Immutability makes that historical distinction visible instead of rewriting the evidence.

Dictionary entries and 26 synthetic examples form separate read-only views. Existing recorded read-aloud and Collection audio can be played when available. No account name or privileged editor is required for the public read-and-analyze workflow.

24 Application Architecture and Public Deployment

Separation of responsibilities

Layer	Responsibility
React / TypeScript	Five-screen UI, anonymous capability bootstrap, input handling, read-only references and display of immutable tests.
FastAPI boundary	Bounded requests, origin/CSRF policy, public allowlist, explicit errors and shared-workspace context.
Lexer and parser	Regex/lexicon classification, language clues, CFG transformations, FIRST/FOLLOW, LL(1) table and bounded parsing.
Shared storage	SQLite-backed Collection, saved profile/grammar, immutable test snapshots, retained ownership and media references.
Operator facilities	Restricted environment, persistent storage, backups, controlled reference maintenance and deployment.

The anonymous bootstrap is not identity authentication. Visitors can create tests, but Collection, grammar and recording mutations are rejected server-side. New analyses must use the saved grammar; a stale grammar yields an explicit 409/reload response. A request UUID supports idempotent retries without duplicating an already-completed test. A deliberately new test counts separately.

Historical account/admin/import code and data remain for compatibility. Their continued existence does not expose those routes publicly. CSRF tokens, secure production cookies and rate limits constrain requests; they do not make public submissions confidential. Public ownership presentation is redacted without changing historical ownership in storage.

Selected hosted topology

Nginx serves HTTPS for <https://camfranglais.duckdns.org> on the Ubuntu VPS at 109.199.120.38. It proxies to the application on 127.0.0.1:2021; port 2021 is private, not an additional public entry point. The restricted systemd service uses persistent data outside the public assets. SMTP and Google credentials are not required by this public mode.

The documentation check observed an HTTP 200 public bootstrap with the correct read-only capabilities and a working no-login browser interface on 27 September. Detailed source-grounded class, sequence, activity, component, deployment and other UML views are provided in the separate SDD and atlas.

25 Verification, Reproducibility and Limitations

Checks and the dates to which they apply

Evidence	Observed result
27 September corpus checks	Seven targeted grammar/persistence tests passed; the exact twelve cases are exercised within them. Seven additional constructed controls in the report generator also matched their expected outcomes.
26 September application release	186 backend/launcher/legacy-storage tests, 148 frontend unit tests and 68 mocked browser tests passed. App/test TypeScript checks and the production build passed.
Live workflow qualification	Fourteen distinct cases passed across the full run and focused rerun. Two full-run startup/teardown interruptions later passed in a three-case mobile rerun; no uninterrupted 14/14 run is claimed.
Documentation reproducibility	Generated tables are compared byte-for-byte with freshly computed evidence; source hashes and the public corpus snapshot are retained. UML/PDF checks verify current figures, class coverage, activations, references and layout.

Known regression

The existing test `GrammarNotationTests.test_every_real_lexer_category_is_a_supported_terminal` still expects an older category set. It omits nine categories now supported by the lexer. A deprecation warning about the installed FastAPI/Starlette test-client transport also appeared during the targeted run without failing it.

Reproduction procedure

Use the configured Python environment and installed project requirements. Run `python -m tools.build_report_evidence -check`, then the targeted grammar/persistence tests. Rebuilding uses the existing local PlantUML/LaTeX task and `python -m tools.check_documentation`. The JSON records complete traces and unabridged grammar names; production numbers in this report are derived from that ordering.

The small development corpus, broad FF/PID categories, unresolved reference ambiguity and two UNKNOWN forms remain substantive limitations. No latency, throughput, uptime or linguistic-accuracy benchmark is claimed.

26 Conclusion

The project demonstrates a compiler pipeline. It preserves all twelve supplied transcriptions, classifies all 81 token occurrences, computes frequency and language clues, transforms a corpus-derived grammar and exposes the complete conflict-free LL(1) table. Successful and unsuccessful traces show how category decisions propagate into parsing.

The two rejected cases are informative. They identify precise unresolved forms at token six. The known-word controls also reject invalid category orders, proving that parsing is more than a dictionary-membership check. Acceptance of nonstandard wording illustrates why the application must keep grammar fit separate from linguistic correctness and supplied meaning.

The public interface removes irrelevant account management while retaining server-side boundaries: anyone can analyze and save an immutable test, but reference data and grammar cannot be changed through the public app. The separate engineering documents describe the classes and interactions.

Priorities after this report

Review the two unknowns and disputed lexical labels using human evidence, evaluate an independent corpus, address the stale category regression, and rehearse the required ten-minute demonstration.

Author group

Group members and matricules:

Member	Matricule
Kwete Ngouba Junior Rayan	ICTU20241377
Djemtchimo Noukui Bruno Jonatan	ICTU20241585
Amina Boubakary	ICTU20241870

References and Evidence Register

- [1] Faculty of Information and Communication Technologies. *CS4110 Compiler Construction, Summer 2026, SET A: Lexical and Syntactic Analysis of Informal Urban Communication in Yaoundé*. Supplied assignment brief; instructor Engr. Tanwi Nkiambo; assessment dated 29 September 2026. Source of the 10–15-statement, group-of-three and 25–30-page requirements.
- [2] Augustin Emmanuel Ebongue. “Linguistic Features in a Marginal Corpus. The Case of Written Camfranglais, a Cameroon Youth Language.” *Multilingual Margins*, 9(1), issue year 2022; published online 25 May 2023. Publisher metadata and abstract checked 27 September 2026. <https://www.epubs.ac.za/index.php/mm/article/view/1392>.
- [3] Alfred V. Aho, Monica S. Lam, Ravi Sethi and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*, second edition. Pearson/Addison-Wesley. General compiler background for lexical analysis, grammar transformations and predictive parsing; not the source of this corpus.
- [4] Camfranglais project source, baseline a1b88aa. `compiler/lexer/tokenizer.py`, `compiler/lexer/languages.py`, `compiler/lexer/reference.py`, `compiler/parser/yaounde.py`, `compiler/parser/analysis.py`, `compiler/parser/predictive.py`. The implemented rules, not textbook examples, determine the reported outputs.
- [5] Project Collection, twelve user-supplied transcriptions. `docs/evidence/final-corpus-20260927.json`. Sanitized read-only snapshot; supply date 22 September, snapshot date 27 September 2026. Original fieldwork provenance remains unconfirmed. French meanings are supplied annotations.
- [6] Reproducible coursework analysis. `tools/build_report_evidence.py`, `docs/evidence/final-analysis-20260927.json`, `docs/report-data/`. Contains source fingerprints, full tokens, sets, table, transformations, traces and boundary controls. Tested against the twelve exact cases in `compiler/tests/yaounde_cases.py`.
- [7] Supplied classified vocabulary and retained reference senses. `dictionary/full_lexicon_classified.csv`, `dictionary/camfranglais.md`, `dictionary/extra_lexicon.md`. Source-labelled lookup material; 878 supplied CSV rows and 938 dictionary display entries, not fieldwork observations.
- [8] Public application screenshots, captured 27 September 2026. `docs/evidence/screenshots/`; <https://camfranglais.duckdns.org>. Existing records were inspected without posting new tests. The capture register records image hashes, context and the saved-test crop.

Supplementary SRS, SDD and UML atlas sources and PDFs are distributed in the same documentation directory.